

Compte-Rendu : TD3 - Requêtes préparées et association de classes (SQL Join)

Matisse KRA
BTS SIO

Compte-Rendu : TD3 - Requêtes préparées et association de classes (SQL Join)	1
Introduction : Présentation des objectifs du TD	1
Exercice 1 : Les requêtes préparées (getVoitureByImmat)	1
Exercice 2 : Enregistrer un objet en base avec save()	1
Exercice 3 : Brancher le formulaire de création de voiture sur la BDD	1
Exercice 4 : Création des tables de notre site de covoiturage	1
Exercice 5 : Premier lien entre utilisateur et trajet	1
Exercice 6 : Association entre utilisateurs et trajets (table passager)	1
Exercice 7 : Liste des utilisateurs d'un trajet (findPassagers)	1
Exercice 8 : Liste des trajets d'un utilisateur (findTrajets)	1
Exercice 9 : Désinscrire un utilisateur d'un trajet (deletePassager)	1
Partie 6 : Créez une injection SQL (non réalisée)	1
Conclusion	1

Introduction : Présentation des objectifs du TD

Ce TD3 est le prolongement du TD2 sur la persistance des données en PHP avec PDO. L'objectif est double. Il s'agit d'abord de sécuriser les accès à la base de données en remplaçant les requêtes construites par concaténation de chaînes par des requêtes préparées, qui protègent des injections SQL. Il s'agit ensuite de modéliser et de coder les associations entre plusieurs tables : un conducteur pour un trajet, et une association non bornée entre utilisateurs et trajets passant par une table pivot. Ce compte-rendu détaille les requêtes préparées (SELECT, INSERT, DELETE), la création des tables et des contraintes de clé étrangère en CASCADE, et les jointures SQL (INNER JOIN) encapsulées dans les classes métiers.

Exercice 1 : Les requêtes préparées (getVoitureByImmat)

J'ai copié dans un nouveau dossier TD3 les fichiers Conf.php, Model.php, voiture.php et lireVoiture.php du TD2. J'ai ensuite ajouté dans la classe Voiture une méthode publique statique getVoitureByImmat(\$immat). Cette méthode n'écrit plus l'immatriculation directement dans la chaîne SQL : elle place un tag :nom_tag dans la requête, la prépare avec prepare(), puis passe les valeurs dans un tableau à execute(). C'est ce mécanisme qui empêche l'injection SQL, car la valeur est transmise séparément de la requête et n'est jamais interprétée comme du code SQL. L'appel à setFetchMode(PDO::FETCH_CLASS, 'Voiture') transforme ensuite le résultat en objet Voiture, et la fonction renvoie false lorsqu'aucune ligne ne correspond. J'ai testé les deux cas dans lireVoiture.php : une immatriculation existante (AB123CD) et une immatriculation inconnue (ZZ999ZZ).

Toutes les voitures (getAllVoitures)

Marque: peugeot
Couleur: rouge
Immatriculation: AB123CD

Exercice 1 : recherche par immatriculation (requête préparée)

Recherche de AB123CD

Marque: peugeot
Couleur: rouge
Immatriculation: AB123CD

Recherche de ZZ999ZZ

Aucune voiture trouvée pour l'immatriculation ZZ999ZZ

Exercice 2 : Enregistrer un objet en base avec save()

J'ai ajouté une méthode d'instance save() dans la classe Voiture. Elle insère l'objet courant (\$this) grâce à la requête INSERT INTO voiture (immatriculation, marque, couleur) VALUES (:immat_tag, :marque_tag, :couleur_tag), elle aussi préparée, les trois valeurs étant fournies à execute() dans un tableau. Contrairement à un SELECT, un INSERT ne renvoie aucun résultat : je n'appelle donc pas fetch() après l'exécution, sous peine de déclencher l'erreur SQLSTATE[HY000]: General error. J'ai testé la méthode dans lireVoiture.php en créant un objet Voiture ('Renault', 'bleu', 'CL789VE') puis en l'enregistrant. La vérification dans phpMyAdmin montre bien la nouvelle ligne dans la table voiture.

Matisse
kra

Toutes les voitures (getAllVoitures)

Marque: peugeot
Couleur: rouge
Immatriculation: AB123CD

Marque: Renault
Couleur: bleu
Immatriculation: CL789VE

Exercice 1 : recherche par immatriculation (requête préparée)

Recherche de AB123CD

Marque: peugeot
Couleur: rouge
Immatriculation: AB123CD

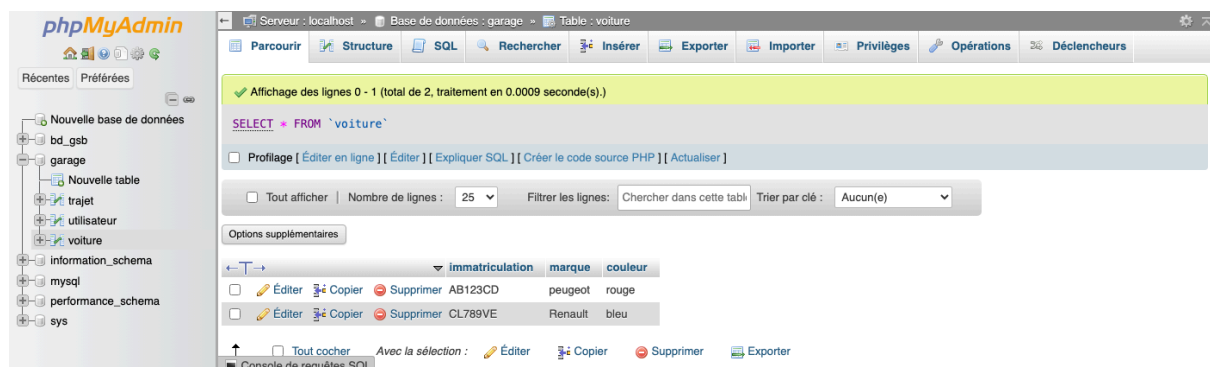
Recherche de ZZ999ZZ

Aucune voiture trouvée pour l'immatriculation ZZ999ZZ

Exercice 2 : enregistrement d'une voiture (save)

La voiture est déjà en base :
Marque: Renault
Couleur: bleu
Immatriculation: CL789VE

Contenu de la table après enregistrement



Exercice 3 : Brancher le formulaire de création de voiture sur la BDD

J'ai copié `creerVoiture.php` et `formulaireVoiture.html` du TD1 dans le dossier TD3. J'ai modifié `creerVoiture.php` pour qu'il inclue `Model.php`, construise l'objet `Voiture` à partir des données reçues en POST, puis appelle `$maVoiture->save()` au lieu de se contenter de l'afficher. J'ai testé l'insertion depuis `formulaireVoiture.html` avec la voiture GT456KL (Citroen, vert), et j'ai vérifié dans phpMyAdmin que la ligne était bien enregistrée. J'ai également protégé tout le code contenant du PDO (`getAllVoitures`, `getVoitureByImmat`, `save`) par des blocs `try - catch`, comme dans `Model.php` : chaque ligne liée à PDO est susceptible de lancer une exception qu'il faut capturer et traiter.

Matisse
kra

Mon formulaire :

Immatriculation :

Marque :

Couleur :

Voiture créée et enregistrée en base :

Marque: Citroen
Couleur: vert
Immatriculation: GT456KL

[Retour au formulaire](#)

The screenshot shows the phpMyAdmin interface. On the left is the database navigation tree with 'garage' selected. The main panel shows the 'Table : voiture' structure. A yellow status bar indicates 'Affichage des lignes 0 - 2 (total de 3, traitement en 0.0017 seconde(s))'. Below it, the SQL query 'SELECT * FROM `voiture`' is shown. A table of data is displayed with columns 'immatriculation', 'marque', and 'couleur'. The data rows are: AB123CD (peugeot, rouge), CL789VE (Renault, bleu), and GT456KL (Citroen, vert). At the bottom, there are buttons for 'Éditer', 'Copier', 'Supprimer', and 'Exporter'.

immatriculation	marque	couleur
AB123CD	peugeot	rouge
CL789VE	Renault	bleu
GT456KL	Citroen	vert

Exercice 4 : Création des tables de notre site de covoiturage

Les tables utilisateur et trajet avaient déjà été créées à la fin du TD2, avec le moteur de stockage InnoDB. J'ai vérifié leur structure : utilisateur (login VARCHAR(32) en clé primaire, nom, prenom) et trajet (id INT en clé primaire auto-incrémenté, depart, arrivee, date, nbplaces, prix, conducteur_login). Une remarque sur l'interclassement : sous MySQL 9, utf8_general_ci n'existe plus sous ce nom, il est devenu utf8mb3_general_ci. J'ai donc conservé utf8mb4_general_ci pour toutes les tables. C'est indispensable pour la suite, car une contrainte de clé étrangère impose des

Matisse
kra

interclassements identiques entre les colonnes liées. J'ai ensuite inséré des utilisateurs, puis quatre trajets sans remplir le champ id afin de laisser jouer l'auto-incrément, et en n'utilisant que des conducteur_login d'utilisateurs existants.

The screenshot shows the phpMyAdmin interface for the 'utilisateur' table. The 'Structure de table' tab is active, displaying the table's schema. The table has three columns: 'login', 'nom', and 'prenom', all of type 'varchar(32)' and using the 'utf8mb4_general_ci' collation. The 'login' column is the primary key. Below the table structure, the 'Index' section shows a primary index on the 'login' column.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
1	login	varchar(32)	utf8mb4_general_ci		Non	Aucun(e)			Modifier Supprimer Plus
2	nom	varchar(32)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus
3	prenom	varchar(32)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus

Action	Nom de l'index	Type	Unique	Compressé	Colonne	Cardinalité	Interclassement	Null	Commentaire
Modifier Supprimer	PRIMARY	BTREE	Oui	Non	login	5	A	Non	

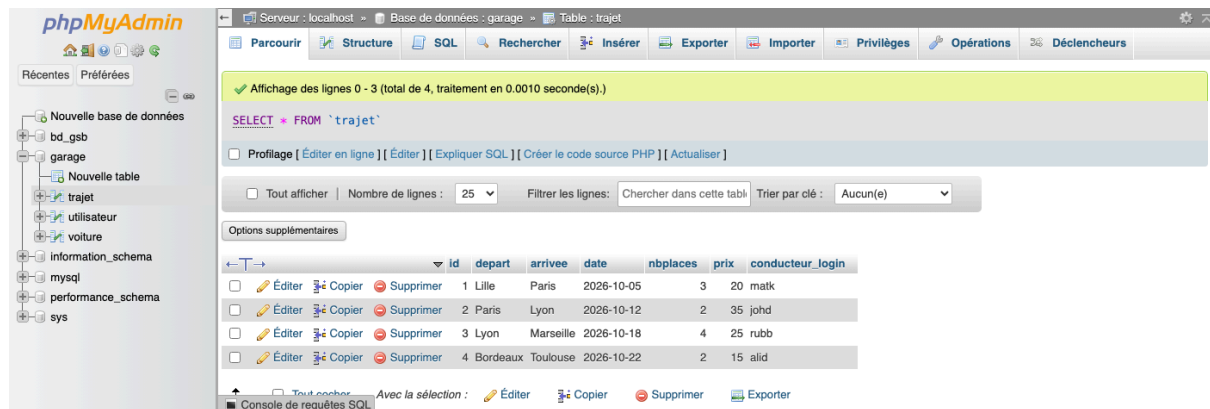
The screenshot shows the phpMyAdmin interface for the 'utilisateur' table, displaying the data tab. A message indicates that 5 lines are displayed. The data table shows five rows of user information.

	login	nom	prenom
☐ Éditer ☐ Copier ☐ Supprimer	alid	dupont	alice
☐ Éditer ☐ Copier ☐ Supprimer	bmarr	martin	bob
☐ Éditer ☐ Copier ☐ Supprimer	johnd	john	doe
☐ Éditer ☐ Copier ☐ Supprimer	matk	matisse	kra
☐ Éditer ☐ Copier ☐ Supprimer	rubb	ruben	batochio

The screenshot shows the phpMyAdmin interface for the 'trajet' table. The 'Structure de table' tab is active, displaying the table's schema. The table has seven columns: 'id', 'depart', 'arrivee', 'date', 'nbplaces', 'prix', and 'conducteur_login'. The 'id' column is the primary key and has an auto-increment attribute. The 'conducteur_login' column is a foreign key that references the 'login' column of the 'utilisateur' table.

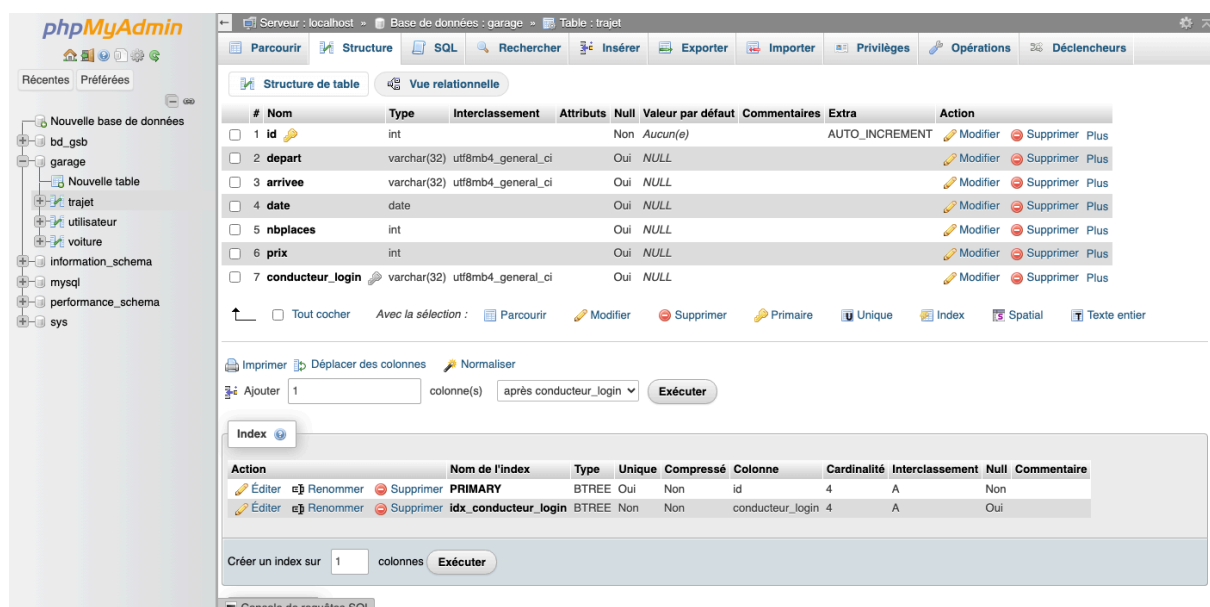
#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
1	id	int			Non	Aucun(e)		AUTO_INCREMENT	Modifier Supprimer Plus
2	depart	varchar(32)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus
3	arrivee	varchar(32)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus
4	date	date			Oui	NULL			Modifier Supprimer Plus
5	nbplaces	int			Oui	NULL			Modifier Supprimer Plus
6	prix	int			Oui	NULL			Modifier Supprimer Plus
7	conducteur_login	varchar(32)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus

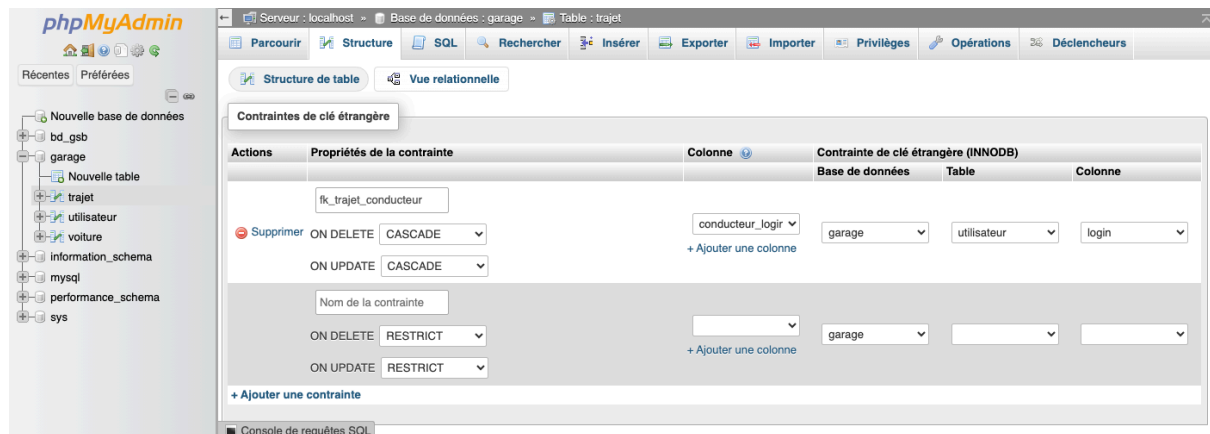
Action	Nom de l'index	Type	Unique	Compressé	Colonne	Cardinalité	Interclassement	Null	Commentaire
Modifier Supprimer	PRIMARY	BTREE	Oui	Non	id	4	A	Non	



Exercice 5 : Premier lien entre utilisateur et trajet

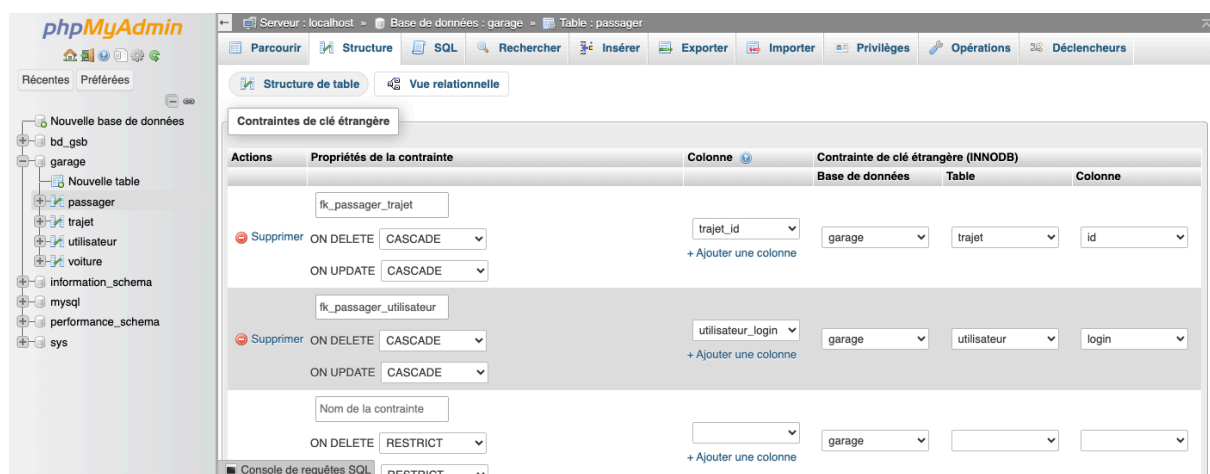
L'association conducteur est de multiplicité 1 : il n'y a qu'un conducteur par trajet. Elle est donc implémentée par le simple champ trajet.conducteur_login. Pour garantir que ce champ corresponde à tout moment à un utilisateur.login existant, j'ai d'abord fait de conducteur_login un index depuis l'onglet Structure de la table trajet. Un index est obligatoire pour une clé étrangère, car MySQL a besoin de retrouver rapidement les lignes concernées pour vérifier la contrainte. J'ai ensuite ajouté la contrainte de clé étrangère vers utilisateur.login via la Vue relationnelle, en choisissant ON DELETE CASCADE et ON UPDATE CASCADE : si un utilisateur est supprimé, ses trajets le sont aussi, et si son login est modifié, la modification est répercutée.



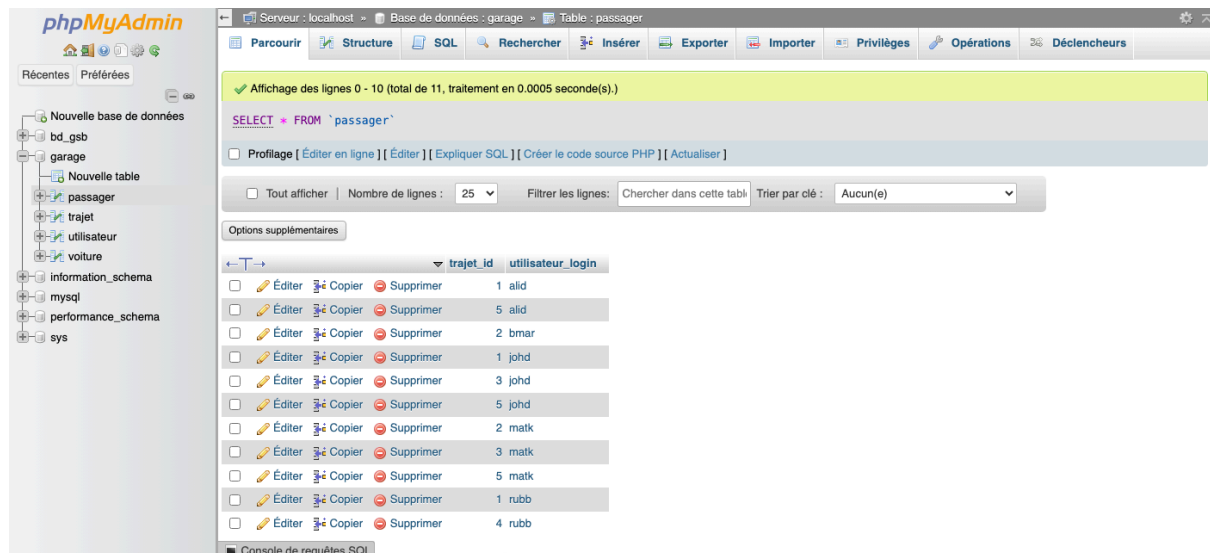


Exercice 6 : Association entre utilisateurs et trajets (table passager)

L'association passager n'est pas bornée : un trajet accueille plusieurs passagers et un utilisateur peut être passager de plusieurs trajets. Je l'ai donc modélisée par une table pivot passager contenant `trajet_id` (INT) et `utilisateur_login` (VARCHAR(32)). La clé primaire est le couple (`trajet_id`, `utilisateur_login`) : prendre `trajet_id` seul limiterait chaque trajet à un unique passager. J'ai ajouté les deux contraintes de clé étrangère, vers `trajet.id` et vers `utilisateur.login`, en `ON DELETE CASCADE` et `ON UPDATE CASCADE`, puis inséré des inscriptions. Pour vérifier concrètement le `CASCADE`, j'ai créé un utilisateur `testc`, un trajet dont il est le conducteur, et trois passagers sur ce trajet. En supprimant `testc` de la table `utilisateur`, son trajet a été supprimé en cascade, et les lignes `passager` de ce trajet ont disparu à leur tour : la table est passée de 11 à 8 lignes.

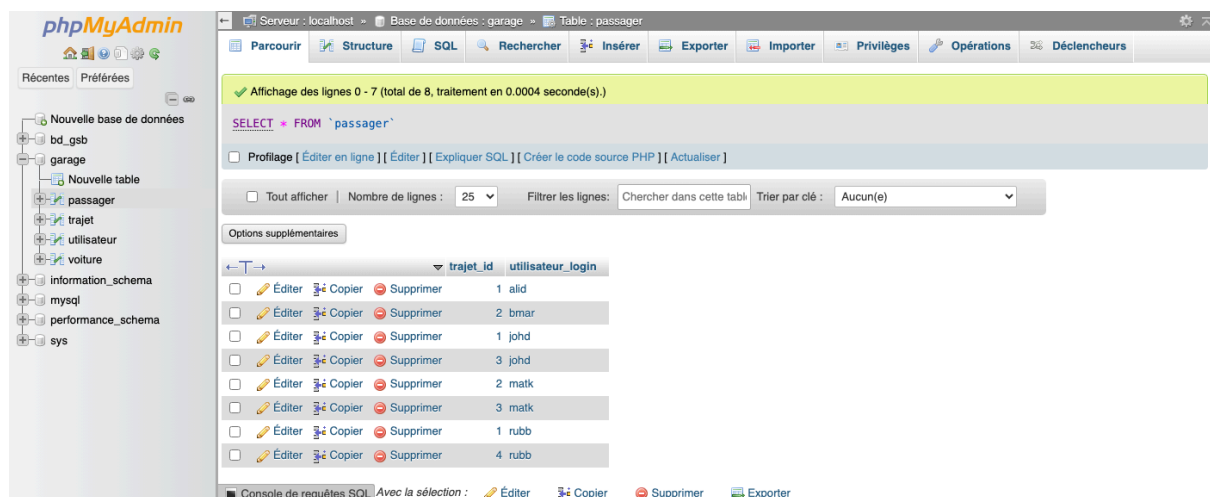


Matisse
kra



phpMyAdmin interface showing the 'passager' table. The table has 11 rows. The columns are 'trajet_id' and 'utilisateur_login'.

trajet_id	utilisateur_login
1	alid
5	alid
2	bmar
1	johd
3	johd
5	johd
2	matk
3	matk
5	matk
1	rubb
4	rubb



phpMyAdmin interface showing the 'passager' table. The table has 8 rows. The columns are 'trajet_id' and 'utilisateur_login'.

trajet_id	utilisateur_login
1	alid
2	bmar
1	johd
3	johd
2	matk
3	matk
1	rubb
4	rubb

Exercice 7 : Liste des utilisateurs d'un trajet (findPassagers)

J'ai d'abord mis à jour la classe Utilisateur pour qu'elle ait les mêmes attributs que la table (login, nom, prenom) et un constructeur à paramètres optionnels. Le test if (lis_null(...)) dans ce constructeur est indispensable : PDO::FETCH_CLASS remplit les attributs de l'objet puis appelle le constructeur, qui sans ce test écraserait par des NULL les valeurs venant de la base. J'ai rencontré exactement ce problème au premier essai, où les passagers s'affichaient vides. J'ai ensuite créé Trajet::findPassagers(\$id), qui fait un INNER JOIN entre utilisateur et passager sur utilisateur.login = passager.utilisateur_login, filtré par passager.trajet_id, et retourne un tableau d'objets Utilisateur. La page testFindUtil.php charge les classes nécessaires, appelle la fonction et affiche chaque utilisateur avec sa méthode

Matisse
kra

afficher(). Le formulaire formFindUtil.php envoie l'identifiant du trajet en GET vers cette page.

Passagers d'un trajet

Trajet numéro 1

Login: alid
Nom: dupont
Prénom: alice

Login: johd
Nom: johne
Prénom: doe

Login: rubb
Nom: ruben
Prénom: batochio

[Retour au formulaire](#)

Passagers d'un trajet :

Identifiant du trajet :

Passagers d'un trajet

Trajet numéro 3

Login: johd
Nom: johne
Prénom: doe

Login: matk
Nom: matisse
Prénom: kra

[Retour au formulaire](#)

Exercice 8 : Liste des trajets d'un utilisateur (findTrajets)

J'ai créé la jointure inverse Utilisateur::findTrajets(\$login) dans Utilisateur.php. La requête fait un INNER JOIN entre trajet et passager sur trajet.id = passager.trajet_id, filtré cette fois par passager.utilisateur_login, et retourne un tableau d'objets Trajet. C'est la même table pivot, lue dans l'autre sens. J'ai créé la page de test testFindTraj.php et le formulaire formFindTraj.php, construits sur le même modèle que ceux de l'exercice 7.

Trajets d'un utilisateur

Utilisateur matk

Départ: Paris
Arrivée: Lyon
Date: 2026-10-12
Nombre de places: 2
Prix: 35
Login du conducteur: johd

Départ: Lyon
Arrivée: Marseille
Date: 2026-10-18
Nombre de places: 4
Prix: 25
Login du conducteur: rubb

[Retour au formulaire](#)

Trajets d'un utilisateur

Utilisateur johd

Départ: Lille
Arrivée: Paris
Date: 2026-10-05
Nombre de places: 3
Prix: 20
Login du conducteur: matk

Départ: Lyon
Arrivée: Marseille
Date: 2026-10-18
Nombre de places: 4
Prix: 25
Login du conducteur: rubb

Exercice 9 : Désinscrire un utilisateur d'un trajet (deletePassager)

J'ai ajouté `Trajet::deletePassager($data)`, qui reçoit un tableau associatif contenant `$data['trajet_id']` et `$data['utilisateur_login']`. La méthode exécute un `DELETE FROM` `passager` préparé portant sur les deux colonnes de la clé primaire, ce qui supprime exactement une inscription. Comme pour l'`INSERT`, un `DELETE` ne renvoie pas de résultat : j'utilise `rowCount()` pour savoir si une ligne a réellement été supprimée et afficher le message correspondant. Le formulaire `formDelPassager.php` demande un identifiant de trajet et un login d'utilisateur, et `testDelPassager.php` effectue la désinscription puis réaffiche les passagers restants du trajet. J'ai désinscrit `alid` du trajet 1 : il ne reste ensuite que `johd` et `rubb`.

Matisse
kra

Désinscrire un passager d'un trajet :

Identifiant du trajet :

Login du passager :

Désinscription d'un passager

L'utilisateur alid a bien été désinscrit du trajet 1.

Passagers restants du trajet 1

Login: johd
Nom: johne
Prénom: doe

Login: rubb
Nom: ruben
Prénom: batochio

[Retour au formulaire](#)

Partie 6 : Créez une injection SQL (non réalisée)

Cette partie optionnelle proposait de simuler une injection SQL sur une table voiture2, à partir des fichiers voiture2.sql et formGetImmatSQL.php fournis avec le sujet. Ces deux fichiers ne faisaient pas partie des documents dont je disposais, je n'ai donc pas réalisé cette manipulation. Le principe reste celui décrit en début de TD : la version vulnérable de getVoitureByImmat concatène directement l'immatriculation dans la requête, si bien qu'une valeur saisie dans le formulaire peut fermer la chaîne et faire exécuter du SQL arbitraire. C'est précisément ce que les requêtes préparées mises en place aux exercices 1 à 3 rendent impossible.

Conclusion

Par rapport au TD2, ce TD apporte deux choses. D'abord la sécurité : toutes les requêtes qui reçoivent une valeur venant de l'extérieur (formulaire, URL) passent désormais par des requêtes préparées, ce qui rend les injections SQL impossibles puisque les valeurs ne sont plus interprétées comme du code SQL. Ensuite la modélisation des associations : les contraintes de clé étrangère en CASCADE garantissent la cohérence de la base, et la table pivot passager permet de représenter une association non bornée entre utilisateurs et trajets, qu'un simple

Matisse
kra

champ supplémentaire ne suffisait pas à décrire. Les jointures SQL correspondantes sont encapsulées dans des méthodes statiques des classes métiers (findPassagers, findTrajets, deletePassager), exactement comme getAllVoitures au TD2. Le code reste ainsi organisé par entité, chaque classe portant ses propres accès aux données, ce qui prépare le passage à une architecture MVC plus propre.